

SPRINT 4

NIVELL 1

Exercici 1: Consulta sobre Taula no Optimitzada (Diagnòstic)

```
1 SELECT *
2 FROM sprint3_silver.companies_clean c
3 JOIN sprint3_silver.transactions_clean t
4   ON c.company_id = t.business_id
5 WHERE c.country = "Germany" AND DATE(t.timestamp) = '2022-03-12' AND t.declined = 0;
```

✓ Esta consulta procesará 11.86 MB cuando se ejecute.

```
7 SELECT *
8 FROM sprint3_silver.companies_clean c
9 JOIN sprint3_silver.transactions_clean t
10  | ON c.company_id = t.business_id
11 WHERE t.declined = 0;
```

✓ Esta secuencia de comandos procesará 23.72 MB cuando se ejecute.

Aquí he fet una comparació on es veu que amb el WHERE es processa exactament la mateixa quantitat de dades ($11,86 + 11,86 = 23,72$) que sense ell ja que abans de filtrar ja ha hagut de llegir totes les dades. Això es pot remediari fent una partició.

Exercici 2: Re-arquitectura i Optimització de l'Emmagatzematge (Partition & Cluster)

```
1 CREATE OR REPLACE TABLE `sprint3_silver.transactions_recent` AS
2 SELECT * EXCEPT(timestamp), TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL CAST(RAND() * 50 AS INT64) DAY) AS timestamp
3 FROM `sprint3_silver.transactions_clean`;
```

✓ Se completó la consulta

```
1 CREATE OR REPLACE TABLE `sprint3_gold.fact_transactions_optimized`
2 PARTITION BY DATE(timestamp)
3 CLUSTER BY business_id AS
4 SELECT *
5 FROM `sprint3_silver.transactions_recent`;
```

✓ Se completó la consulta

Tipo de tabla	Particionada
Particionada por	DAY
Particionada en el campo	timestamp
Vencimiento de la partición	60 días
Filtro de partición	No es obligatorio
Agrupada por	business_id

En primer lloc, utilitzem el EXCEPT per seleccionar tot menys “timestamp” per a que no surti repetit. Després, a la nova columna que hem generat utilitzem: CURRENT_TIMESTAMP() per seleccionar el dia d’avui; RAND() * 50 per seleccionar un número a l’atzar entre 0 i 50 (que li fem CAST AS INT64 per a que sigui un número enter, ja que no es poden restar 5,3 dies); INTERVAL DAY per seleccionar que són intervals de dies; i tot això dins de TIMESTAM_SUB per restar els dies.

Amb el PARTITION BY el que estem fent és separant la taula per dies i amb

CLUSTER BY fem que cada partició s’agrupi per “business_id”.

Exercici 3: La Prova del Cotó (Benchmark)

```
1 SELECT *
2 FROM sprint3_silver.transactions_recent
3 WHERE DATE(timestamp) >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY) AND declined = 0;
```

✓ Esta consulta procesará 11.85 MB cuando se ejecute.

```
1 SELECT *
2 FROM sprint3_gold.fact_transactions_optimized
3 WHERE DATE(timestamp) >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY) AND declined = 0;
```

✓ Esta consulta procesará 6.72 MB cuando se ejecute.

La consulta optimitzada processa un 56,71% de la informació en relació amb la que no està particionada ni agrupada. Té bastant sentit aquest resultat, ja que 30 de 50 és el 60%. En conclusió, si fem la partició de les taules podem estalviar MB, que en aquest cas estalviem un 43,29%.

Exercici 4: Smart Caching (Vistes Materialitzades)

```
1 CREATE OR REPLACE MATERIALIZED VIEW `sprint3_gold.mv_daily_sales` AS
2 SELECT DATE(timestamp) AS dia, SUM(amount) AS diners, COUNT(transaction_id) AS transaccions
3 FROM `sprint3_gold.fact_transactions_optimized`
4 GROUP BY dia;
```

✓ Esta consulta procesará 0 B cuando se ejecute.

```
1 SELECT dia, ROUND(diners, 2) AS diners, transaccions
2 FROM sprint3_gold.mv_daily_sales;
```

✓ Esta consulta procesará 1.2 KB cuando se ejecute.

Fila	dia	diners	transaccions
1	2026-08-09	252429.56	974
2	2026-09-28	266845.89	1057
3	2026-08-15	487453.42	1905
4	2026-09-02	487797.38	1908
5	2026-09-15	485888.57	1923

Podem afirmar que les vistes materialitzades processen poca informació, encara que la facturació mínima per consulta són 10MB

NIVELL 2

Exercici 1: Perfilat de Clients VIP (Mètriques Agregades amb CTEs)

```
1 WITH VIP_Stats AS (  
2   SELECT user_id,SUM(amount) AS total_gastat,COUNT(transaction_id) AS num_compres,ROUND(AVG(amount), 2) AS tiquet_mig, MAX(amount) AS  
   max_compra  
3   FROM `sprint3_gold.fact_transactions_optimized`  
4   GROUP BY user_id  
5   HAVING total_gastat > 500  
6 )  
7 SELECT v.user_id,CONCAT(u.name, ' ', u.surname) AS nom_complet,u.email,v.num_compres,v.tiquet_mig,v.max_compra,ROUND(v.total_gastat, 2) AS  
   total_gastat  
8 FROM VIP_Stats v  
9 JOIN `sprint3_silver.users_combined` u  
10  ON CAST(v.user_id AS STRING) = u.user_id  
11 ORDER BY v.total_gastat DESC;
```

✓ Esta consulta procesará 5.4 MB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

>I

Resultados de la consulta

Chat

View all jobs

Guardar los resultados

Abrir en

Información del trabajo

Resultados

Visualización

JSON

Detalles de la ejecución

Gráfico de ejecución

Fila	user_id	nom_complet	email	num_compres	tiquet_mig	max_compra	total_gastat
1	289	Dxwgi Hwcru	dxwgi.hwcru@example.com	94	441.35	554.71	41486.54
2	318	Bnyr Astuw	bnyr.astuw@example.com	91	443.39	615.51	40348.5

Amb el WITH es crea la CTE. Al SELECT posem tot el que es demana, s'agrupa per "user_id" i es filtra amb el HAVING els que s'han gastat més de 500. Després, es junta aquesta taula amb "users_combined" canviant en datatype de "user_id" de la CTE perquè no era el mateix que el de l'altre taula.

Exercici 2: Anàlisi de Tendències (Window Functions sobre Vistes)

```
1 SELECT dia AS Data, ROUND(diners, 2) AS Vendes_Avui, ROUND(LAG(diners) OVER (ORDER BY dia), 2) AS Vendes_Ahir,  
2   ROUND((diners - LAG(diners) OVER (ORDER BY dia)) / LAG(diners) OVER (ORDER BY dia) * 100, 2) AS Diff_Percentual  
3 FROM `sprint3_gold.mv_daily_sales`  
4 ORDER BY dia ASC;
```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

> Resultados de la consulta [Chat](#) [View all jobs](#) [Guardar los resultados](#)

Información del trabajo		Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico de ejecución
Fila	Data	Vendes_Avui	Vendes_Ahir	Diff_Percentual		
1	2026-08-09	252429.56	null	null		
2	2026-08-10	546262.53	252429.56	116.4		
3	2026-08-11	511655.81	546262.53	-6.34		
4	2026-08-12	501386.66	511655.81	-2.01		
5	2026-08-13	538177.93	501386.66	7.34		

A aquesta consulta estem utilitzant la funció LAG() i OVER(). Amb el LAG() el que fem és agafar la fila anterior, i amb l'OVER() les estem ordenant per dies, i així agafar el dia d'abans.

També, per fer el "Diff_Percentual" estem restant les vendes d'avui menys les d'ahir, això ho dividim per les vendes d'ahir i ho multipliquem per 100.

A la primera fila, a "Vendes_ahir" i "Diff_Percentual" el valor és NULL perquè no hi ha dades un dia anterior.

Exercici 3: Totals Acumulats (Running Totals sobre Vistes)

```
1 SELECT dia AS Data, ROUND(diners, 2) AS Vendes_del_Dia,
2     ROUND(
3     SUM(diners) OVER (
4     PARTITION BY EXTRACT(YEAR FROM dia)
5     ORDER BY dia
6     ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
7     ), 2) AS Vendes_Acumulades_YTD
8 FROM `sprint3_gold.mv_daily_sales`
9 ORDER BY dia ASC;
```

✓ Esta consulta procesará 816 B cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

> Resultados de la consulta

Información del trabajo		Resultados	Visualización
Fila	Data	Vendes_del_Dia	Vendes_Acumula...
1	2026-08-09	252429.56	252429.56
2	2026-08-10	546262.53	798692.09
3	2026-08-11	511655.81	1310347.9
4	2026-08-12	501386.66	1811734.56
5	2026-08-13	538177.93	2349912.49

En primer lloc, per fer l'acumulació de diners i que es reinici cada any, estem partint la taula per anys, per això hem utilitzat, EXTRACT(YEAR FROM dia), per extraure l'any de la data. Dins d'aquesta partició ho ordenem per dies, i amb ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW fem que la suma vagi desde la fila actual fins la primera sense límits dins de la partició.

Exercici 4: Fidelització i Valor del Client (Filtratge Avançat)

```
1 WITH Ranking AS (
2     SELECT user_id, timestamp, amount,
3     ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY timestamp ASC) AS num,
4     AVG(amount) OVER (
5     PARTITION BY user_id
6     ORDER BY timestamp ASC
7     ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
8     ) AS mitj_3
9 FROM `sprint3_gold.fact_transactions_optimized`
10 QUALIFY ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY timestamp ASC) <= 3
11 )
12 SELECT u.user_id, CONCAT(u.name, ' ', u.surname) AS nom_complet, u.email, r.timestamp AS data_3a_compra, r.amount AS import_3a_compra,
13     ROUND(r.mitj_3, 2) AS mitjana_3_primeres
14 FROM Ranking r
15 JOIN `sprint3_silver.users_combined` u
16 ON CAST(r.user_id AS STRING) = u.user_id
17 WHERE r.num = 3
18 ORDER BY r.mitj_3 DESC;
```

Información del trabajo		Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico de ejecución	
Fila	user_id	nom_complet	email	data_3a_compra	import_3a_compra	mitjana_3_primei	
1	1383	Zmvwbx Bdzpdwyc	zmvwbx.bdzpdwyc@example.c...	2026-08-29 16:05:06.046985 UTC	477.17	534	
2	3057	Sudzhl Hqlwlpkb	sudzhl.hqlwlpkb@example.com	2026-08-28 16:05:06.046985 UTC	608.29	534	
3	3252	Hfwbhi Lijiiiah	hfwbhi.lijiiiah@example.com	2026-08-18 16:05:06.046985 UTC	697.25	521	
4	3219	Jwoujf Ryfftavu	jwoujf.ryfftavu@example.com	2026-08-14 16:05:06.046985 UTC	645.67	523	
5	997	Uxhpfr Nqljwbek	uxhpfr.nqljwbek@example.com	2026-08-18 16:05:06.046985 UTC	423.23	521	
6	4330	Jshqsf Bcpeksoz	jshqsf.bcpeksoz@example.com	2026-08-17 16:05:06.046985 UTC	340.25	516	

Aquí primer utilitzem el WITH per fer una CTE fem servir ROW NUMBER() per crear una columna que va enumerant les files segons l'ordre que li diem respectant la partició. Després, fer AVG() de la fila amb les 2 anterior (2 PRECEDING AND CURRENT ROW) i amb el QUALIFY ens quedem amb les que a la columna “num” ens quedem amb els 3 primers només.

Per últim, juntem aquesta CTE amb “users_combined” i la filtrem només agafant les files de “num” 3, que és aquesta mitjana la que necessitem.

NIVELL 3

Exercici 1: Desanidament i Aplanament de Dades (Unnesting)

```
1 CREATE OR REPLACE TABLE `sprint3_gold.dim_transactions_flat` AS
2 SELECT t.transaction_id,t.timestamp,t.amount AS total_ticket,p.product_id AS product_sku,p.name AS product_name,p.price AS product_price
3 FROM `sprint3_gold.fact_transactions_optimized` t
4 CROSS JOIN UNNEST(t.product_ids) AS single_product_id
5 JOIN `sprint3_silver.products_clean` p
6 ON CAST(single_product_id AS STRING) = CAST(p.product_id AS STRING);
```

Se completó la consulta

Esquema	Detalles	Vista previa	Estadísticas	Linaje	Perfil de datos	Calidad de los datos	
Fila	transaction_id	timestamp	total_ticket	product_sku	product_name	product_price	
1	873E7298-B7E7-4832-8097-16C...	2026-08-09 16:05:06.046985 UTC	189.38	66	mustafar jinn	2.12	
2	F8342CFA-F0EF-4E64-A84E-D2F...	2026-08-09 16:05:06.046985 UTC	196.41	66	mustafar jinn	2.12	
3	3A0394CF-AB67-40E5-B9A7-2A...	2026-08-09 16:05:06.046985 UTC	61.88	66	mustafar jinn	2.12	
4	B1289883-0D74-41DF-8A91-11...	2026-08-09 16:05:06.046985 UTC	331.04	66	mustafar jinn	2.12	
5	38220B2C-6476-487B-9853-36F...	2026-08-09 16:05:06.046985 UTC	350.23	66	mustafar jinn	2.12	
6	74C511A3-EE32-4C38-AE8E-02...	2026-08-09 16:05:06.046985 UTC	340.94	66	mustafar jinn	2.12	
7	47A43501-43CD-48A1-BACA-1F...	2026-08-09 16:05:06.046985 UTC	400.01	66	mustafar jinn	2.12	
8	42260C74-A7B9-4E3B-B5DF-66...	2026-08-09 16:05:06.046985 UTC	223.86	66	mustafar jinn	2.12	
9	9C544933-1C06-4ED4-BDE8-EB...	2026-08-09 16:05:06.046985 UTC	2.12	66	mustafar jinn	2.12	
10	85035559-41C0-43AE-A984-E7...	2026-08-09 16:05:06.046985 UTC	319.69	66	mustafar jinn	2.12	
11	21B39E45-D964-465E-A216-F6...	2026-08-09 16:05:06.046985 UTC	113.17	66	mustafar jinn	2.12	
12	736DA969-6E95-402E-AC1C-C0...	2026-08-09 16:05:06.046985 UTC	2.12	66	mustafar jinn	2.12	
13	9C2E2F7E-4CA2-4348-96A7-DA...	2026-08-09 16:05:06.046985 UTC	315.93	66	mustafar jinn	2.12	

[Esquema](#)[Detalles](#)[Vista previa](#)[Estadísticas](#)[Linaje](#)[Perfil](#)

Filtro Escribir el nombre o valor de la propiedad

☐	Nombre del campo	Tipo	Modo	Descripción
☐	transaction_id	STRING	NULLABLE	-
☐	timestamp	TIMESTAMP	NULLABLE	-
☐	total_ticket	FLOAT	NULLABLE	-
☐	product_sku	STRING	NULLABLE	-
☐	product_name	STRING	NULLABLE	-
☐	product_price	FLOAT	NULLABLE	-

Aquí el més destacat que hem fet és utilitzar UNNEST per trencar l'ARRAY, és a dir, en aquest cas l'ARRAY és a product_id, i el que hem fet és que cada product_id es separés i estigués a una nova fila. Amb el CROSS JOIN el que fem és que aquestes noves files creades tinguin la resta d'elements que tenien a la seva fila.

Exercici 2: El Rànquing de Vendes (Agregació Simple)

```
1 SELECT product_name, COUNT(*) AS unitats_venudes, ROUND(SUM(product_price), 2) AS facturacio_total
2 FROM `sprint3_gold.dim_transactions_flat`
3 GROUP BY product_name
4 ORDER BY unitats_venudes DESC
5 LIMIT 5;
```

✓ Esta consulta procesará 5.99 MB cuando se ejecute.

Fila	product_name	unitats_venudes	facturacio_total
1	Winterfell	10071	872608.39
2	the duel	10059	1053379.53
3	dooku solo	10022	574416.39
4	Karstark Dorne	7694	704138.02
5	skywalker ewok	7607	1271917.33

He agrupat per producte per contar les unitats venudes i els diners que s'han fet amb aquest producte, ho he ordenat de més a menys venudes i he limitat a 5 per a que només surtin els 5 més venuts.

Exercici 3: Automatització del Pipeline i Visualització

```
1 CREATE OR REPLACE FUNCTION `sprint3_gold.calculate_tax`(preu NUMERIC, iva NUMERIC)
2 RETURNS NUMERIC AS (
3 | ROUND(preu * (1 + iva), 2)
4 );
```

Se completó la consulta

```
1 CREATE OR REPLACE TABLE `sprint3_gold.dim_transactions_flat` AS
2 SELECT t.transaction_id,t.timestamp,t.amount AS total_ticket,p.product_id AS product_sku,p.name AS product_name,p.price AS product_unit_price,
3 | `sprint3_gold.calculate_tax`(CAST(p.price AS NUMERIC), 0.21) AS product_price_tax_inc
4 FROM `sprint3_gold.fact_transactions_optimized` t
5 CROSS JOIN UNNEST(t.product_ids) AS single_product_id
6 JOIN `sprint3_silver.products_clean` p
7 | ON CAST(single_product_id AS STRING) = CAST(p.product_id AS STRING);
```

Esta consulta procesará 7.09 MB cuando se ejecute.

Exportar como consulta programada

Configuración válida.

Detalles y programación

Nombre de la consulta programada *
taxa

Opciones de programación

Frecuencia de repetición *

Horas

Frecuencia de repetición *

24

horas



☐ Comenzar ahora ☒ Comenzar a la hora definida

Fecha de inicio y hora de ejecución *

30/9/26, 7:00 a.m.

000



☒ Sin hora de finalización programada

☐ Programar una hora de finalización

Fecha de finalización

000



Programado para ejecutarse cada 24 horas a partir del 30 de septiembre de 2026 a las 7:00:00 a.m. 000.

Primer he creat la funció amb dues variables: preu i iva. No multiplico el preu per 1,21 per a que sigui dinàmica i es puguin posar altres valors de taxes. Després, he modificat la taula on he posat la funció al SELECT i així surt com altre columna i he canviat el tipus de data de "price" perquè funcioni la funció, i aquí sí que he posat l'iva del 21%. A l'hora de programar la consulta m'ha sortit un problema que hi ha a l'imatge de sota i no la puc programar.

Se produjo un error mientras se creaba la consulta programada:
Billing is disabled for this project. Please ask the project owner to enable it.

<https://datastudio.google.com/reporting/0e0fafb6-89ad-495e-aea5-9d94cb3e6280/page/wt69F/edit>